

Development Phases

Eleven stages, each one the cheapest possible unblock for the next. No stage begins until the previous one builds, passes its tests, and has been **used by hand**.

This is not an exam app with extra features bolted on.

An exam is one *mode* of a shared preparation engine, alongside practice, interviews and flashcard review. A preparation type — academic exam, interview, certification, coding test — is a row of configuration, never a subclass. That single decision is what lets the platform grow from “NIIT Semester 2” to “AWS Certification” without a rewrite, and every phase below is sequenced to protect it.

STAGES	COMPLETE	MVP ENDS AT	STACK
11	2	Phase 7	Spring Boot 4 · Next 16

At a glance

Effort is measured in focused working sessions, not calendar time. The MVP is the point at which this app replaces every other study tool — everything after it is genuine enhancement rather than completion.

#	STAGE	WHAT IT DELIVERS	EFFORT	STATE
00	Scaffolding	Repo, build, database, both apps running	½ session	DONE
01	Foundation	Auth, preparation spaces, subjects and topics	2–3	DONE
02	Question Engine	Question bank, typed content, import and export	2–3	NEXT
03	Practice Sessions	The session engine and the immutable attempt log	2	MVP
04	Exam Mode	Timed mocks, navigation, results breakdown	2	MVP
05	Adaptive Engine	Weak-topic detection and adaptive difficulty	3	MVP
06	Spaced Repetition	Review scheduling and flashcards	2	MVP
07	Analytics & Readiness	Dashboard and an explainable readiness score	2	MVP
08	AI Layer & Interviews	Generation, explanation, interview follow-ups	3	LATER
09	PWA & Offline	Installable app, offline study, sync	2	LATER
10	Polish & Deploy	Accessibility, performance, security, hosting	2–3	LATER

Why this order, and not another

Each stage exists to make the next one cheap. The question bank precedes the session engine because you cannot test a session with no questions. Exams precede the adaptive engine because mock scores are the yardstick the engine is tuned against. AI comes late because it *writes into* the question bank and *reads from* the learner model — if either is wrong, AI amplifies the fault at scale and at cost. Offline comes last of the feature work because it mirrors a data model that must have stopped changing.

The stages in detail

Each stage states what it is, why it exists in that position, what gets built, and the concrete thing you do by hand to satisfy yourself it works.

00

Scaffolding

½ session

COMPLETE

The unglamorous stage that makes every later one boring in the right way. A repository, a reproducible build, a real database in Docker, and both applications starting from a single documented command.

BUILT	Spring Boot backend with the Maven wrapper committed, so no global Maven install is needed. Next.js frontend. PostgreSQL 16 in Docker with a health check. All secrets in a git-ignored environment file, with a documented template beside it.
VERIFY	The health endpoint returns <code>{"status": "UP"}</code> and both builds pass.

01

Foundation

2–3 sessions

COMPLETE MVP

Identity, and the concept everything else hangs from: the **preparation space**. A space is a self-contained world — its own subjects, its own question bank, its own history, its own readiness. Two spaces share no state and the whole engine. This must be right first because the space identifier is a partition key on roughly eight tables; adding it later would mean touching everything.

BUILT	Registration and sign-in with a short-lived access token plus a rotating refresh cookie. Preparation spaces, and the six preparation types seeded as configuration rows. Subjects, and topics as a tree three levels deep. A single ownership gate every module passes through. Architecture rules enforced as tests rather than as prose.
VERIFY	Build one space with five subjects and nested topics, create a second space of a different type, and confirm it is completely empty. Then try to open the first space while signed in as somebody else: it reports that the space does not exist.

IN PROGRESS

MVP

The content layer. A question carries no user performance data at all — that separation is what keeps question banks shareable, importable and exportable, and it is why the same imported question can live in two spaces with entirely independent histories.

BUILT

Questions with promoted columns for everything you filter on and a validated JSON payload for everything that varies by type, so a new question type needs no schema change. Multiple choice, multiple select, true/false, short answer and flashcards. Editing creates a new version rather than overwriting, so a typo fixed in March never rewrites what you were scored on in January. Import from JSON and CSV with a dry run that reports errors and duplicates before anything is written, and export back out so your material is never trapped in this app.

VERIFY

Paste a CSV containing two deliberate mistakes, watch the dry run catch both, fix and import. Then edit a question and confirm the original version survives.

MVP

The single session engine that every later mode reuses. Practice is simply the first mode implemented; exams, interviews and flashcard reviews are the same machinery with different rules. Building it once here is what stops the app from growing three parallel copies of scoring, history and analytics.

BUILT

Sessions and their items, plus the **attempt log** — append-only and never updated. Answer submission, evaluation per question type, immediate feedback with the explanation, and a session summary. The runner chrome that all later modes inherit: progress, keyboard shortcuts, feedback panel.

WHY APPEND-ONLY

Every derived number in the system — mastery, review schedule, readiness — is rebuildable from this log. That is what makes it safe to improve the algorithms later without losing history.

VERIFY

Run ten questions, get four wrong deliberately, and check the summary and the recorded response times.

MVP

A mock exam under real conditions. Notably this is *not* a new subsystem: it is a session with a deadline, deferred feedback and a fixed blueprint. It also deliberately does *not* adapt — a mock that adjusts itself is not a yardstick you can compare across weeks.

BUILT	Configurable question count and duration. Question navigation grid, mark-for-review, unanswered tracking, and automatic submission at the deadline. Results with a breakdown by subject, by topic and by difficulty, time spent per question, and every incorrect answer with its explanation.
INTEGRITY	The timer is server-authoritative. The browser renders a countdown, but the server records the start and rejects a late submission — otherwise your own scores would mean nothing to you.
VERIFY	Sit a twenty-question, fifteen-minute mock. Mark three for review, leave two blank, let it auto-submit. Close the browser mid-exam and confirm the remaining time is still correct when you return.

MVP

The part that makes this not a quiz app. It learns what you are weak at and serves you that, at a difficulty pitched to stretch you without breaking you. It is kept deliberately free of the framework and the database so the algorithm can be rewritten and tested against a hand-built learner in milliseconds.

BUILT	An ability estimate per topic that updates after every answer, and a matching difficulty rating per question that corrects itself — so a question you mislabelled fixes itself after a handful of responses. Question selection scored on review debt, weakness, difficulty fit, syllabus coverage and freshness. Weak-topic detection, and a rebuild job that replays the attempt log from scratch.
DIFFICULTY	Targets roughly a 78% success rate — high enough to stay motivating, low enough to meet real resistance. Difficulty is relative to you, so a “hard” question quietly becomes a medium one as you improve, with nobody relabelling anything.
KINDNESS	Guardrails are a feature, not a safety net: a recovery mode after a bad run, a guaranteed win every few questions, no long runs of the hardest material, and a topic is never called <i>weak</i> on fewer than five attempts — only <i>unassessed</i> .
VERIFY	Fail six React Hooks questions on purpose, then start practice and confirm Hooks dominates and the difficulty steps down rather than piling on.

MVP

Deciding *when* something should come back, so you revise just as you are about to forget. A flashcard here is a question type, not a separate system — which is why Again/Hard/Good/Easy feeds the same engine that drives your exam readiness, for free.

BUILT	A review schedule per item, per space. Ordinary quiz answers are translated into a grade using correctness, response time and confidence — so a lucky guess is not scheduled as though it were solid knowledge. A due queue, review-debt tracking, and a flashcard runner that previews the interval on each button before you press it.
VERIFY	Review ten cards with different grades and confirm the previewed intervals match what is actually scheduled, and that “Again” brings a card back within the session.

MVP

MVP ENDS HERE

The daily answer to “what should I do right now, and am I actually ready?” Readiness is deliberately not your average quiz score: six weighted components covering syllabus coverage, accuracy, depth, retention, consistency and mock performance.

BUILT	A nightly readiness snapshot that stores its own breakdown, so the app can say “ <i>readiness fell three points because fourteen items are overdue — clearing the backlog is worth about four</i> ” rather than showing a number that cannot explain itself. A daily dashboard with the recommended session, prioritised weak topics, streaks, and charts that each answer a study question.
HONESTY	The score is scaled by how much evidence exists. After twelve questions you cannot be “88% ready”, and a system that says so is lying at the exact moment the stakes are highest. Early readiness is capped and labelled as low confidence.
VERIFY	Use it for a week before going further. At this point the app should have replaced whatever you were using instead.

ENHANCEMENT

AI is an enhancement, never a foundation. With no provider configured the application stays fully functional and every test still passes; AI controls simply render disabled with a reason. This is also where interview preparation becomes conversational rather than a quiz in disguise.

BUILT	A provider-agnostic layer so Claude, OpenAI or a local model are interchangeable configuration rather than a rewrite. Question generation from your own notes, explanations of wrong answers, a similar question on the same concept, and short lessons on weak topics. Interview mode proper: an answer evaluated on correctness, completeness, technical accuracy and clarity, followed by a realistic follow-up that responds to what you actually said.
SAFETY	Nothing generated reaches your question bank without passing a review step. That is the difference between AI helping you build a bank and AI quietly corrupting your study data. Cost and latency are logged from the first call, not after the first surprising bill.
VERIFY	Generate fifteen questions from your React notes, reject four, approve the rest. Then disable AI entirely and confirm nothing else breaks.

ENHANCEMENT

Installable on your phone and usable without a connection, because study happens on buses and in waiting rooms. Comes late because it mirrors a data model that must have stopped changing.

BUILT	Installable app with an icon and offline shell. A downloadable question pack per space. Practice and flashcard review offline with local scoring, and answers queued for upload. Because attempts are append-only events with a client-generated identifier, syncing is conflict-free by construction — the worst case is a duplicate the server ignores.
VERIFY	Airplane mode: complete twenty questions and fifteen cards offline, reconnect, and confirm all thirty-five land exactly once.

ENHANCEMENT

Making what exists genuinely good rather than merely working, and putting it somewhere you can reach it from any device.

BUILT	A key board-only pass over every session mode, focus management and contrast. Query and bundle performance. A dependency and secrets audit, and tests for token rotation and reuse. End-to-end tests over the exam, practice, flashcard and interview runners. Backup, full data export, production deployment and a runbook.
-------	---

The rules that hold across every stage

These are not aspirations written in a document nobody re-reads. Most are enforced by tests that fail the build, or by constraints the database itself refuses to violate.

- **An exam is a mode, not an entity.** One session engine serves practice, exams, interviews and flashcards. The moment an exam becomes a first-class thing, everything cross-cutting gets written three times.
- **A preparation type is a row, not a subclass.** Adding “PMP Certification” is an insert. Adding a genuinely new format costs one small strategy class, not a parallel copy of the session, scoring and analytics stack.
- **Questions carry no performance data.** Review state is keyed by user, space and question, so the same imported question can live in two spaces with independent histories — and question banks stay shareable.
- **The attempt log is append-only.** Every derived number can be thrown away and rebuilt from it, which is what makes improving the algorithms safe rather than destructive.
- **Nothing is scoped by accident.** The preparation space identifier is carried on every per-space table and is a required parameter on every query. Cross-space references are rejected by the database, not merely discouraged in review.
- **Access failures look like absence.** Requesting somebody else's space returns “not found”, never “forbidden”, so the API cannot be used to discover what exists.
- **The engine is testable without a database.** Selection and scheduling take plain values in and return plain values out. A build-failing rule enforces it from before the first line of that engine was written.
- **Nothing is deleted that history references.** Archiving hides; it does not destroy months of your own study record.
- **Secrets live in the environment.** No key, token or connection string belongs in the source, and the application refuses to start with a placeholder signing key.

The governing requirement

Build the foundation so this can evolve from an exam preparation tool into a general-purpose adaptive preparation platform **without requiring a rewrite**. Every decision in every stage above was chosen against that single test.